Surface Crack Detection in Historical Buildings with Deep Learning-based YOLO Algorithms: A Comparative Study

Article *in* Computational Research Progress in Applied Science and Engineering · September 2024

DOI: 10.61186/crpase.10.3.2904

CITATION
1

READS
522

3 authors, including:

Hasan Ali Akyürek
Necmettin Erbakan University
25 PUBLICATIONS **28** CITATIONS

SEE PROFILE

Hasan Ibrahim KOZAN
Necmettin Erbakan University
31 PUBLICATIONS **15** CITATIONS

SEE PROFILE

COMPUTATIONAL RESEARCH PROGRESS IN APPLIED SCIENCE & ENGINEERING (CRPASE)

CRPASE: TRANSACTIONS OF
ELECTRICAL, ELECTRONIC AND COMPUTER ENGINEERING

Journal homepage: http://www.crpase.com

CRPASE: Transactions of Electrical, Electronic and Computer Engineering 10 (3) Article ID: 2904, 1–14, September 2024 ISSN 2423-4591

Research Article

Surface Crack Detection in Historical Buildings with Deep Learning-based YOLO Algorithms: A Comparative Study

Hasan Ali Akyürek [1], Hasan İbrahim Kozan*[2], Şakir Taşdemir [3]

[1] *Department of Avionics Engineering, Faculty of Aviation & Space Sciences, Necmettin Erbakan University, Konya, Turkiye*
[2] *Department of Food Processing, Meram Vocational School, Necmettin Erbakan University, Konya, Turkiye*
[3] *Department of Computer Engineering, Faculty of Technology, Selcuk University, Konya, Turkiye*

Keywords

Surface Crack Detection,
Historical Buildings,
YOLO Algorithms,
Deep Learning,
Cultural Heritage
Preservation.

Abstract

The identification of surface cracks is critically important for regular buildings, but this significance increases substantially for historical structures. Traditionally, these cracks are identified and detected through processes that are time-consuming, prone to human error, and labor-intensive. This study introduces a novel approach by investigating the use and performance of multiple deep learning-based YOLO algorithms to detect surface cracks specifically in historical structures, which has not been extensively explored in previous research. A comprehensive dataset containing 4,912 images of stone, brick, tile, and concrete from historical sites was used for training and testing. The dataset was chosen due to its diversity in material types and its relevance to historical preservation efforts. Performance evaluations were conducted using YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10, assessing them in terms of speed and robustness. The results indicate that YOLOv9 achieved the highest accuracy, with an accuracy rate of 92.1% and a processing speed of 1.051 hours per model training.

1. Introduction

Cultural heritage is crucial for the advancement of civilizations. Historical buildings are among the most physically prominent of these heritages. These structures, constructed long ago using various methods and techniques, have survived to the present day, either wholly or partially. The architectural details of such buildings can be intricate, requiring specialized expertise for the repair of potential damage. However, detecting such damage also requires special effort and expertise. Over time, aging, environmental effects, natural disasters, human interactions, and organic reactions can damage the entire structure or certain parts of it. In a study, researchers observed deformations caused by these interactions in ceramic tiles. Such damages can pose a risk to the structure itself, potentially harming people in the event of falling or fragmentation, shortening the structural lifespan, and degrading the overall appearance of the building [1].

To mitigate these adverse effects, various traditional methods have been developed over time. These methods, primarily considered as preventive measures, include examples such as buttresses and support elements like corner keys, horizontal keys, tie beams, and anchor tie beams. These measures, which involve multiple factors such as material compatibility, consolidation, and timber connection details, serve as basic examples [2]. Despite these preventive measures or the lack of need for similar measures in certain materials or structures of the time, cracks that form can tend to cause more severe damage. Until now, traditional methods

* Corresponding Author: Hasan İbrahim Kozan
E-mail address: hkozan@erbakan.edu.tr, ORCID: https://orcid.org/0000-0002-2453-1645

Received: 07 August 2024; Revised: 10 August 2024; Accepted: 12 August 2024
https://doi.org/10.61186/crpase.10.3.2904

Academic Editor: **Mubarak Adesina**

Please cite this article as: H.A. Akyürek, H.İ. Kozan, Ş. Taşdemir, Surface Crack Detection in Historical Buildings with Deep Learning-based YOLO Algorithms: A Comparative Study, Computational Research Progress in Applied Science & Engineering, CRPASE: Transactions of Electrical, Electronic and Computer Engineering 10 (2024) 1–14, Article ID: 2904.

such as expert mapping or visual scanning of the structure have been used to detect these cracks [3]. Nowadays, non-destructive methods such as photogrammetry, laser scanning, or qualitative IRT, and quantitative methods like heat flux meters, quantitative IRT, tracer gas, or fan pressurization methods are being developed [4], but they have not yet reached a sufficient and desired level [5]. Computer vision-based approaches, including various non-destructive methods, have gained significant interest due to their ability to provide clear visual evidence of damage in images. Techniques such as fast Haar transform, FFT, Sobel edge detector, and Canny edge detector have been used for crack detection in RGB images [6, 7]. Additionally, non-destructive methods like Hough transform [8] and wavelet analysis [9] have been employed to extract damage-sensitive features related to different types of structural damage in these images. Different electromagnetic sensors [10], IR methods [11], hybrid image scanning with deep learning algorithms [12], fiber optic sensors [13-15] or distributed optical fiber sensors [15] or camera-based systems [16, 17] are also popular as non-destructive techniques.

The preservation and value of cultural heritage have increased in recent years due to the accelerated pace of environmental and climatic changes, the proliferation of structures, and the intensified effects of natural disasters. Studies[18] indicate that the loss of such historical values can rapidly increase due to impacts like earthquakes. Many factors contribute to the deformation of such important structures, with cracks in the structures or elements being one of them. These defects can result from the mentioned reasons, along with chemical reactions, natural processes over time, weak mortar materials, and factors such as expansion and contraction [19].

Indeed, a study [20], has indicated that structural defects can arise from factors causing cracks and noted that these cracks impact fundamental load-bearing characteristics such as strength and durability. Particularly, it has been reported that deep cracks in structures or parts that have stood for a long time, regardless of the cause, can potentially increase damage and cause problems in certain parts or the entirety of the related cultural heritage, leading to a loss of integrity [21].

As is well known, human errors are relatively more frequent compared to technology-centered models. Attempting to detect cracks manually through visual inspection implies a subjective identification of the cracks. This situation not only brings the disadvantage of subjectivity but also results in a very time-consuming and costly process. Another issue with such manual methods of crack detection is the subjective interpretation of the crack's depth. This generally results in insufficient identification of the crack's characteristics, which can produce irreversible structural outcomes [22]. The complete evacuation of Joshimath, a town in India, due to widespread crack formation, can be considered another indication of the importance of crack detection.

As previously mentioned, although various traditional methods based on sensors, such as plaster bridges, fissure meters, or general crack indicators exist, these and other traditional methods may be inapplicable, prohibitively expensive, or yield inaccurate results in cases where the structure is very large, highly sensitive, or impossible to damage. Even in other non-destructive methods, factors such as material/equipment requirements, feasibility, the suitability of modular devices to the environment, and the sensitivity and balance of the environment must be considered. While these methods can produce excellent results in detecting crack depths and patterns, practical disadvantages can still be observed [23, 24].

All these reasons highlight the need for newer techniques. One of the most prominent and critical issues globally is focusing on resolving defects that may arise over time in the fastest and most cost-effective manner. Therefore, regarding the preservation of cultural heritage, it is undoubtedly most exciting and effective to achieve real-time and accurate results in crack detection. In this context, machine learning techniques, particularly deep learning (DL), emerge as a promising solution. Machine learning is one of the most suitable technological approaches for solving such issues. In recent years, a machine learning technique developed for real-time results, YOLO (You Only Look Once), has achieved significant success in detecting cracks in such structures [25]. YOLO, which can be used not only in buildings considered historical and cultural heritage but also in modern buildings, is crucial for detecting cracks whose occurrence and timing cannot be predicted. Minimizing human errors, protecting cultural heritage, preventing potential harm to people, and producing practical results without logistical needs make YOLO very prominent in this regard. The YOLO machine learning algorithm, developed over a long period, is progressing rapidly, with its use increasing swiftly across 10 different versions. While each version has its own benefits, some also lead to increases in required resources or highlight certain deficiencies. This study aims to use YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10 algorithms to solve this problem, identify differences between these algorithms, and evaluate speed, performance data, accuracy, and resource requirements in crack detection. For this purpose, each algorithm was analyzed meticulously using the same hardware and platforms with similar parameters.

2. Related work

Crack detection in concrete structures and buildings is crucial for assessing risks to the safety, maintenance, and integrity of buildings. Traditional inspection methods, which are typically based on manual assessments, are time-consuming and subjective, leading to potential inaccuracies in identifying structural defects. To overcome these challenges, researchers have turned to advanced technologies, especially deep learning algorithms, for automatic crack detection.

YOLO algorithms have been used in a variety of fields beyond crack detection, demonstrating their adaptability and effectiveness. To help create safer urban environments, traffic management systems have integrated YOLO to identify cars and pedestrians in real-time [26]. Furthermore, enhanced YOLO models specifically designed to recognize small UAVs in challenging environments are becoming increasingly relevant in drone detection applications, enhancing airspace security [27]. Another noteworthy use is for baggage security, where YOLOv5 has been used to

identify forbidden goods during airport inspections, proving that it is effective in high-stakes situations [28].

YOLOv5 has been successfully used to detect and categorize cracks in concrete structures, demonstrating its capability to detect complex backgrounds and various crack sizes. A study has highlighted the importance of using deep learning models to increase the sensitivity and effectiveness of crack detection in urban environments [5].

This study aims to provide a detailed comparison by evaluating the success of YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10 in detecting cracks in various types of structures. By analyzing the strengths and weaknesses of each algorithm, this research will contribute to the development of more efficient and reliable crack detection systems and lead to applications that will increase the safety and longevity of both historical and modern buildings.

3. Materials and methods

3.1. Dataset

In this study, we used an organized dataset designed for crack detection, gathered from various locations and materials. This dataset includes images of stones and bricks captured at historic bridges in Isfahan, cob images from historical buildings in desert villages around Isfahan, and tile images from structural ornaments in northern Portuguese cities such as Porto, Barcelos, Braga, and Guimarães. Additionally, photos of concrete bridges in Isfahan were also included.

The dataset, which consists of a total of 4,912 images after initial collection and subsequent cleaning, has been documented in detail and used in a recent publication [5]. The original dataset creators meticulously cleaned the data by removing low-quality or irrelevant images to ensure the quality and accuracy of the data used. The dataset was divided into three subsets: 80% for training, 10% for validation, and 10% for testing. Table 1 provides a summary of dataset.

Table 1. Selected dataset information

Categories	No. of images	Device used for capturing photos	Resolution	Scaled resolution	Total images
Stone	998	64-megapixel Samsung Galaxy A32 camera	3456 × 3456	416 × 416	4912
Brick	1344				
Tile	241				
Cob	992				
Concrete	1397				

3.2. Hardware and Software

The study was conducted on a workstation with an i9-14900K processor, 16GB RTX-A4000 graphical processing unit and 64GB random access memory, running in a Python-3.11.7 environment.

3.3. You Only Look Once (YOLO) Algorithm

You Only Look Once (YOLO) is an open-source object detection algorithm based on convolutional neural networks. One of the most well-known deep learning algorithms is YOLO, which distinguishes itself from the others with its single-stage detection architecture and speed as shown in Figure 1 [29].

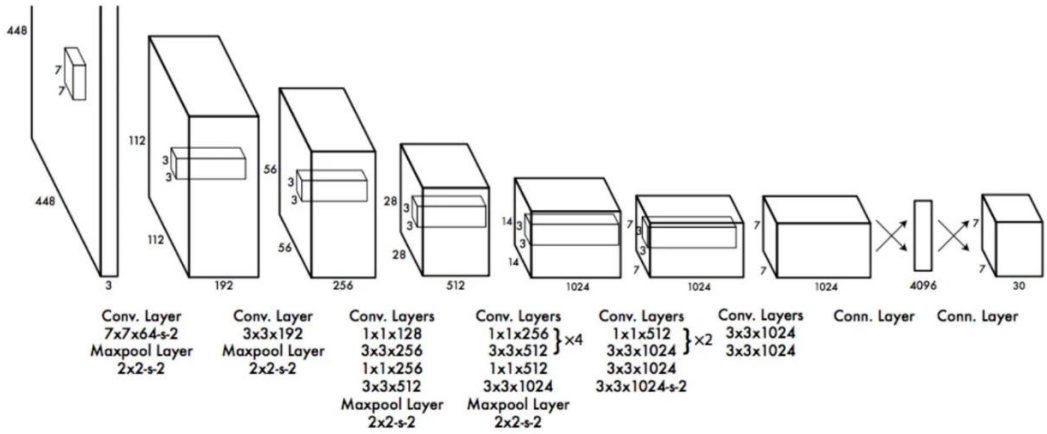

Figure 1. YOLO architecture

YOLO first divides the input picture into grids of SxS for detection. Depending on the version, these grids' sizes may vary; for instance, $3x3$, $5x5$, and $19x19$ grids can be used. The task of each grid within itself is to determine the class, length, height, and presence of an object in the field as well as whether it is within the midpoint. These processes result in the creation of bounding boxes. Next, for every grid, an estimation vector is made. The confidence score, B_x is the object's x coordinate, B_y is the object's y coordinate, B_w is the object's width, B_h is the object's height and the dependent

class probability are all contained in the prediction vector [30].

From YOLOv5 to YOLOv10, the YOLO series got a substantial evolution, with improvements being made to architecture, performance, and efficiency with each iteration. These models are becoming increasingly suitable for real-time industrial applications, as analyses show a trend towards higher accuracy and lower computational costs.

With its efficient processing speeds and CSPNet architecture, YOLOv5 provides a solid foundation. This has been expanded with subsequent iterations such as YOLOv6 and YOLOv7, which offer state-of-the-art features such as enhanced backbone networks and anchor-free detection, further improving performance. This trend was maintained by YOLOv8 and YOLOv9, which prioritized lightweight designs and sophisticated training methods. Finally, YOLOv10 represents a culmination of these advancements, offering state-of-the-art performance metrics and architectural innovations that cater to the demands of modern computer vision tasks. Table 2 summarizes the key characteristics and performance metrics of these YOLO models, providing a clear comparison of their capabilities [31, 32].

Table 2. Comparisons from YOLOv5 to YOLOv10

Model	Release	Architecture	Use cases
YOLOv5	2020	CSPNet for efficiency, SPP for multi-scale feature extraction, anchor-based prediction.	Underwater object detection, real-time video analysis
YOLOv6	2022	Lightweight design, improved feature extraction components.	Real-time applications requiring high speed and moderate accuracy
YOLOv7	2022	ELAN (Efficient Layer Aggregation Network), complex backbone.	High-performance tasks like surveillance and autonomous driving
YOLOv8	2023	Depth wise separable convolutions, multi-head self-attention.	Waste sorting, complex object detection scenarios
YOLOv9	2023	Focus on efficiency and accuracy refinements (specifics less documented)	Enhancing existing use cases and introducing new applications
YOLOv10	2024	Two-stage training, NMS-free training, spatial-channel decoupling	Ideal for real-time applications with high accuracy and low latency

4. Results & Discussions

Table 3 shows scores for YOLO models, including mAP50, mAP50-95, Precision, Recall and training time. The higher the mAP score, the better the accuracy of detecting the target objects [33].

Table 3. Scores of various YOLO models

Model	Epochs	P	R	F1	mAP50	mAP50-95	Train Duration (hours)	Params (m)	Inference (ms)	GPU Mem. (GB)	Weight Size (MB)
YOLOv5	100	0.861	0.834	0.847	0.903	0.621	0.392	2.503	2.9	5.93	5.3
YOLOv6	100	0.878	0.801	0.838	0.894	0.585	0.982	4.630	5.6	9.45	10.2
YOLOv7	100	0.836	0.776	0.805	0.844	0.466	1.108	6.007	5.0	4.99	12.0
YOLOv8	100	0.873	0.837	0.855	0.899	0.646	0.392	3.005	3.0	4.61	6.3
YOLOv9	100	0.929	0.801	0.860	0.921	0.721	1.051	1.970	3.6	3.10	4.7
YOLOv10	100	0.843	0.784	0.812	0.860	0.612	0.757	2.694	2.7	3.18	5.8

Among the different YOLO algorithms, YOLOv9 achieved the best mAP50-95 score of 0.721 and the highest mAP50 score of 0.921. However, it required slightly more time (1.051 hours), placing it second to YOLOv7, which took 1.108 hours.

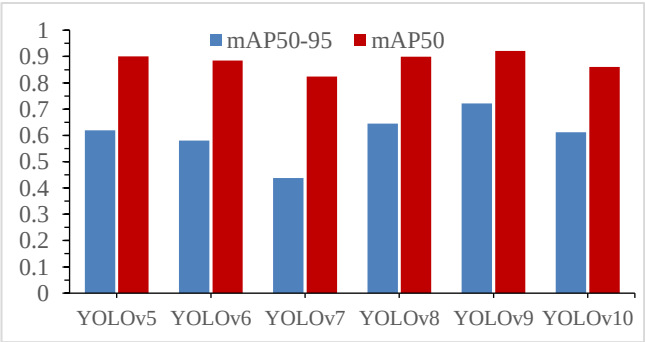

Figure 2. mAP50 and mAP50-95 scores for different YOLO algorithms.

Figure 2 shows a comparison between YOLO algorithms in mAP50 and mAP50-95 scores and YOLOv9 is the highest in both, YOLOv5 still giving good scores with 0.903 for mAP50 directly after YOLOv9 but mAP50-95 is 0.620 in the third place, YOLOv8 mAP50-95 is 0.645 and close to YOLOv9.

The mAP50 scores for comparisons of the YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10 was shown in Figure 3 and for mAP50-95 was shown in Figure 4.

In the comparison of the different YOLO versions, each version exhibits distinct strengths and weaknesses. YOLOv5 starts with a moderate performance (0.2458) and steadily improves, reaching a value of 0.9009 by Epoch 100. YOLOv6, despite showing no performance in the first 19 Epochs, maintains a consistently high level from Epoch 20 onwards, typically around 0.78, and reaches 0.8855 by Epoch 100. YOLOv7, although it begins with a lower

accuracy (0.0819), gradually improves but still lags behind the other versions. YOLOv8 demonstrates strong performance, especially from Epoch 13 onward, surpassing YOLOv5 and peaking at 0.9003 by Epoch 91. YOLOv9 consistently outperforms the other versions, maintaining high accuracy throughout and reaching its peak at 0.9275 in Epoch 83. Finally, YOLOv10 shows steady improvement, reaching its highest accuracy of 0.8604 by Epoch 100. Overall, YOLOv9 stands out as the best-performing model, closely followed by YOLOv8 and YOLOv5, while YOLOv7 falls behind the others.

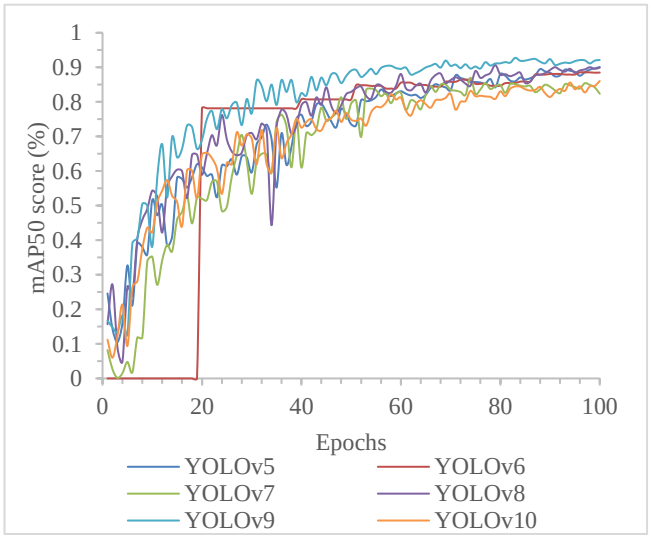

Figure 3. mAP50 scores for different YOLO algorithms during training process.

When comparing the mAP50-95 results across different YOLO versions as shown in Figure 4, distinct patterns of performance emerge. YOLOv5 shows a steady improvement throughout the epochs, beginning with a moderate accuracy of 0.0850 in Epoch 1 and reaching 0.6191 by Epoch 100. YOLOv6, which starts with no performance in the first 19 epochs, shows a sharp increase starting at Epoch 20 and maintains a relatively high accuracy level, peaking at 0.5829 in Epoch 99. YOLOv7, while initially low at 0.0200 in Epoch 1, gradually improves but remains behind the other versions, achieving its highest accuracy of 0.4685 in Epoch 93. YOLOv8 demonstrates consistent improvement, with its mAP50-95 increasing steadily from 0.0518 in Epoch 1 to 0.6453 in Epoch 100, showing strong performance particularly in the later epochs. YOLOv9 consistently outperforms the other versions, starting strong and maintaining high accuracy across the epochs, peaking at 0.7216 in Epoch 100. Lastly, YOLOv10 shows a consistent and gradual improvement, starting from 0.0279 in Epoch 1 and reaching its highest value of 0.6116 by Epoch 100. Overall, YOLOv9 stands out as the best performer in terms of mAP50-95, with YOLOv8 closely following, while YOLOv7 lags behind the others throughout the epochs.

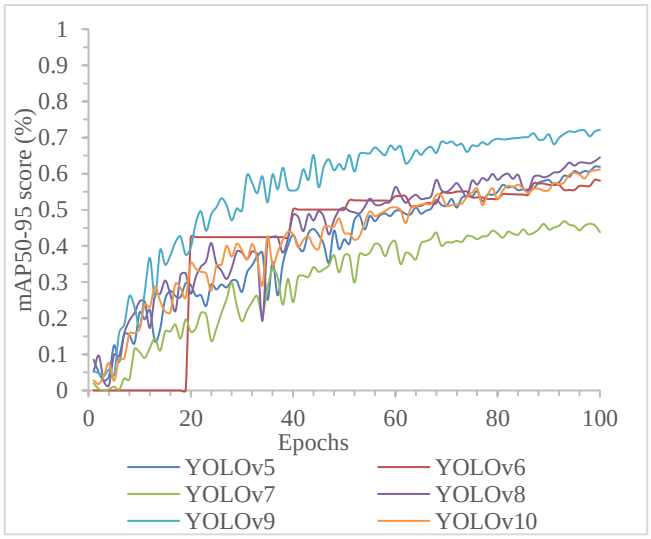

Figure 4. mAP50-95 scores for different YOLO algorithms during training process.

In a study [34], researchers studied on improving YOLOv5 algorithm for crack detection, and in another study [35], researchers focused on road defect detection by using YOLOv5, YOLOv6 and YOLOv7, and a similar study [36] was conducted by using YOLOv5, YOLOv6 and YOLOv7, in another study [37] authors studied on pacement crack identification by using an improved YOLOv8, and in another study [38] researchers have focused on autonomous crack detection, and a similar study [39] was conducted for pavement damage detection by using YOLO8, YOLO9 and YOLO10. The results of the mentioned studies and our own findings are summarized in Table 4, providing a comprehensive comparison.

Table 4. Comparative analysis of YOLO model performance across different studies

Model	Epochs	P	R	F1	mAP50	mAP50-95	Train(hours)	Params(m)	Inference(ms)
YOLOv5(ours)	100	0.861	0.834	0.847	0.903	0.621	0.392	2.503	2.9
YOLOv5 [34]	100	0.620	0.606	0.613	0.633	0.321	N/A	44.0	N/A
YOLOv5 [36]	100	0.77	0.66	N/A	0.71	0.37	N/A	7.03	N/A
YOLOv5-s [35]	100	N/A	N/A	N/A	0.77	N/A	4.42	N/A	<100
YOLOv6(ours)	100	0.878	0.801	0.838	0.894	0.585	0.982	4.630	5.6
YOLOv6 [36]	100	0.75	0.62	N/A	0.72	0.39	N/A	18.207	N/A
YOLOv6s [35]	100	N/A	N/A	N/A	0.681	N/A	5.25	N/A	<100
YOLOv7(ours)	100	0.836	0.776	0.805	0.844	0.466	1.108	6.007	5.0
YOLOv7 [36]	100	0.78	0.72	N/A	0.75	0.36	N/A	6.109	N/A
YOLOv7[34]	100	0.629	0.601	0.615	0.640	0.338	N/A	34.8	N/A
YOLOv7 [35]	100	N/A	N/A	N/A	0.790	N/A	5.78	N/A	<100
YOLOv8(ours)	100	0.873	0.837	0.855	0.899	0.646	0.392	3.005	3.0
YOLOv8 [37]	N/A	0.916	0.785	0.846	0.779	N/A	N/A	N/A	N/A
YOLOv9(ours)	100	0.929	0.801	0.860	0.921	0.721	1.051	1.970	3.6
YOLOv9n [38]	N/A	82.5	76.7	87.4	85.3	N/A	3.05	50.97	N/A
YOLOv10(ours)	100	0.843	0.784	0.812	0.860	0.612	0.757	2.694	2.7
YOLOv10s [39]	200	N/A	N/A	0.560	0,533	N/A	N/A	8.0	N/A

In this study, the performance of YOLO models (YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10) was comparable to or better than that reported in other studies.

For example, our YOLOv5 model achieved a mAP50 of 0.903, while similar studies reported mAP50 values ranging between 0.633 and 0.77. Likewise, our YOLOv6 model achieved a mAP50 of 0.894, compared to mAP50 values in other studies ranging from 0.681 to 0.72. Additionally, our YOLOv7 model recorded a mAP50 of 0.844, which is comparable to or slightly better than the mAP50 values reported in other studies, some of which were as low as 0.640. Our YOLOv8 model achieved a mAP50 of 0.899, while other studies reported values around 0.779, indicating comparable or slightly improved performance. For YOLOv9, we observed a mAP50 of 0.921, which aligns well with or exceeds the performance reported in other works, which showed values up to 0.853. Finally, our YOLOv10 model recorded a mAP50 of 0.860, while similar studies reported values around 0.533, suggesting a better performance in our case.

In terms of parameters (Params), our models were generally optimized with fewer parameters, which contributed to shorter training durations. For instance, our YOLOv5 model had 2.503 million parameters and a training duration of 0.392 hours, whereas other studies reported parameter counts as high as 7.03 million, leading to longer training times.

4.1. YOLOv5 Results

The confusion matrix for YOLOv5 in Figure 5 reveals the number of instances that were correctly classified and misclassified for each class.

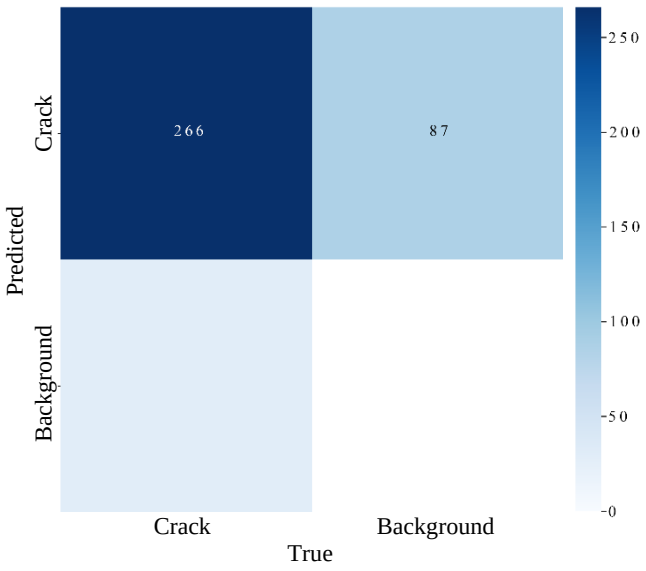

Figure 5. Confusion matrix for YOLOv5 model

Crack class has 266 instances correctly classified, and 87 instances were misclassified as Background.

A detailed description of the YOLOv5 model's 100-epoch training procedure is shown in Figure 6. The model expedites the training process by effectively utilizing parallel processing capabilities. finished in the admirable time of 0.392 hours. The model performs admirably, as evidenced by the final metrics, which include a precision of 0.860, recall of 0.834, mAP50 of 0.903, and mAP50-95 of 0.620. Throughout the epochs, the model consistently reduced its loss values, culminating in final validation box, class, and DFL losses.

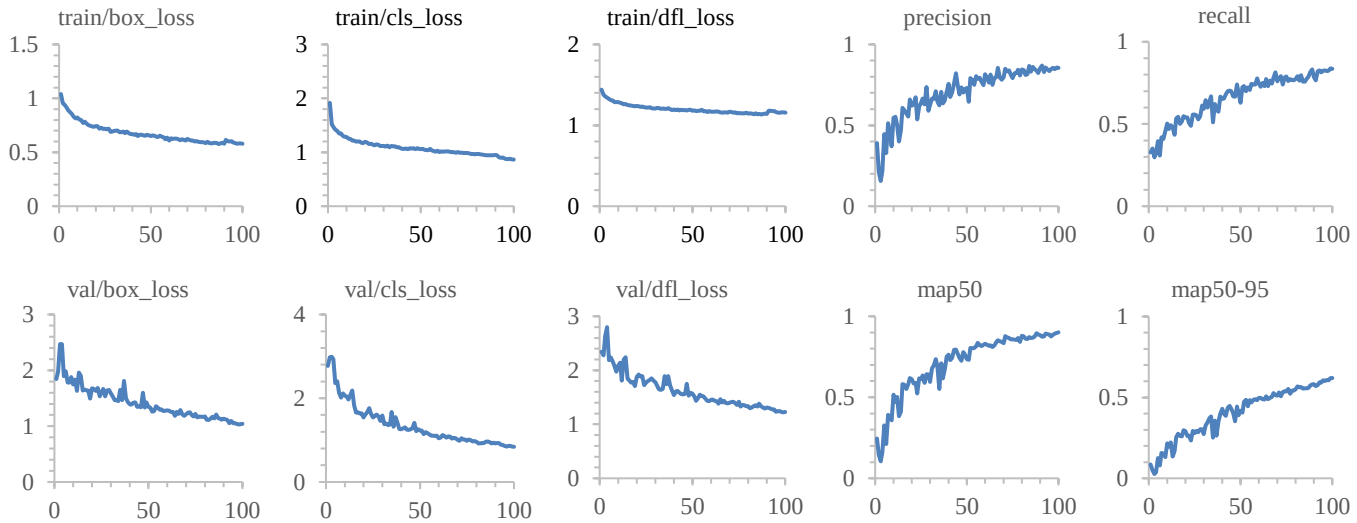

Figure 6. YOLOv5 training and validation results.

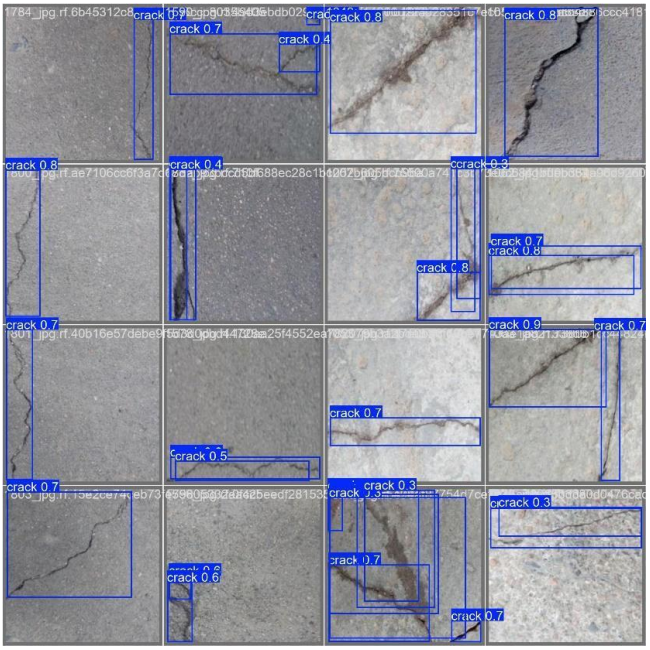

Figure 7. YOLOv5 result sample

Figure 7 shows sample of results for YOLOv5 model and how the model works in real-life.

4.2. YOLOv6 Results

The confusion matrix for YOLOv6 in Figure 8 reveals the number of instances that were correctly classified and misclassified for each class.

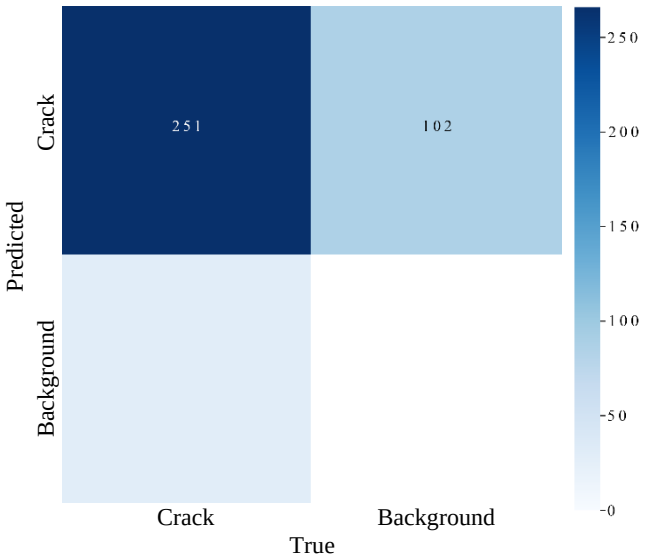

Figure 8. Confusion matrix for YOLOv6 model

Crack class has 251 instances correctly classified, and 102 instances were misclassified as Background.

A detailed description of the YOLOv6 model's 100-epoch training procedure is shown in Figure 9. The model expedites the training process by effectively utilizing parallel processing capabilities. finished in the admirable time of 0.392 hours. The model performs admirably, as evidenced by the final metrics, which include a precision of 0.581, recall of 0.515, mAP50 of 0.884, and mAP50-95 of 0.580. Throughout the epochs, the model consistently reduced its loss values, culminating in final validation box, class, and DFL losses.

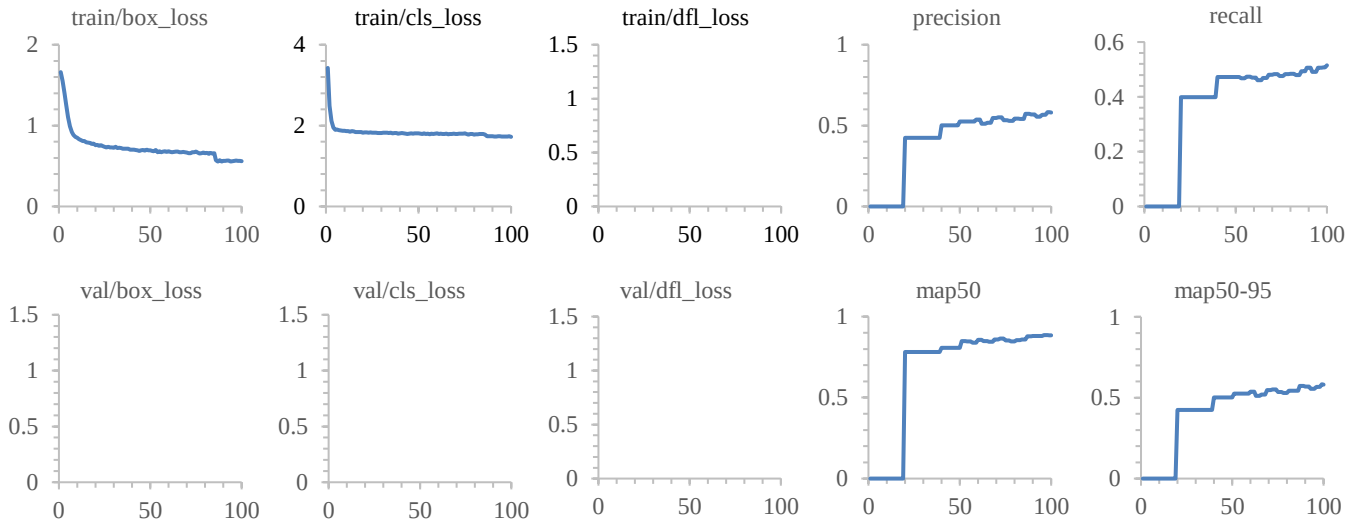

Figure 9. YOLOv6 training and validation results

4.3. YOLOv7 Results

The confusion matrix for YOLOv7 in Figure 10 reveals the number of instances that were correctly classified and misclassified for each class.

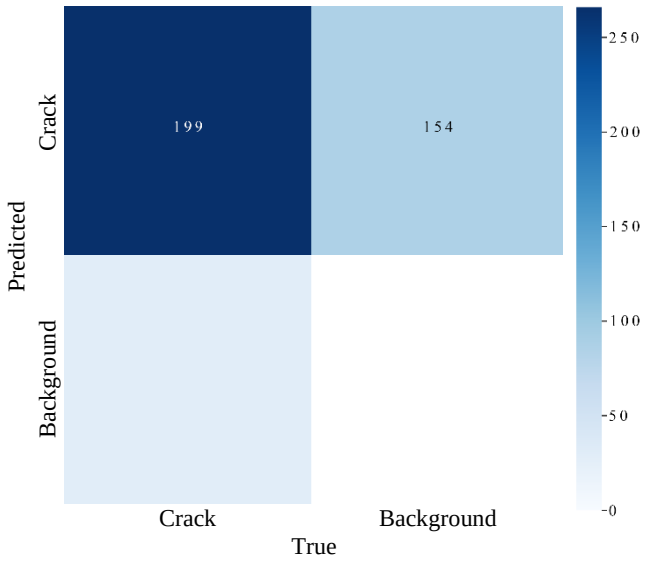

Figure 10. Confusion matrix for YOLOv7 model

Crack class has 199 of instances that correctly classified as Crack, and 154 of instances misclassified as Background.

A detailed description of the YOLOv7 model's 100-epoch training procedure is shown in Figure 11. The model expedites the training process by effectively utilizing parallel processing capabilities. finished in the admirable time of 1.108 hours. The model performs admirably, as evidenced by the final metrics, which include a precision of 0.777, recall of 0.834, mAP50 of 0.823, and mAP50-95 of 0.438. Throughout the epochs, the model consistently reduced its loss values, culminating in final validation box, class, and DFL losses.

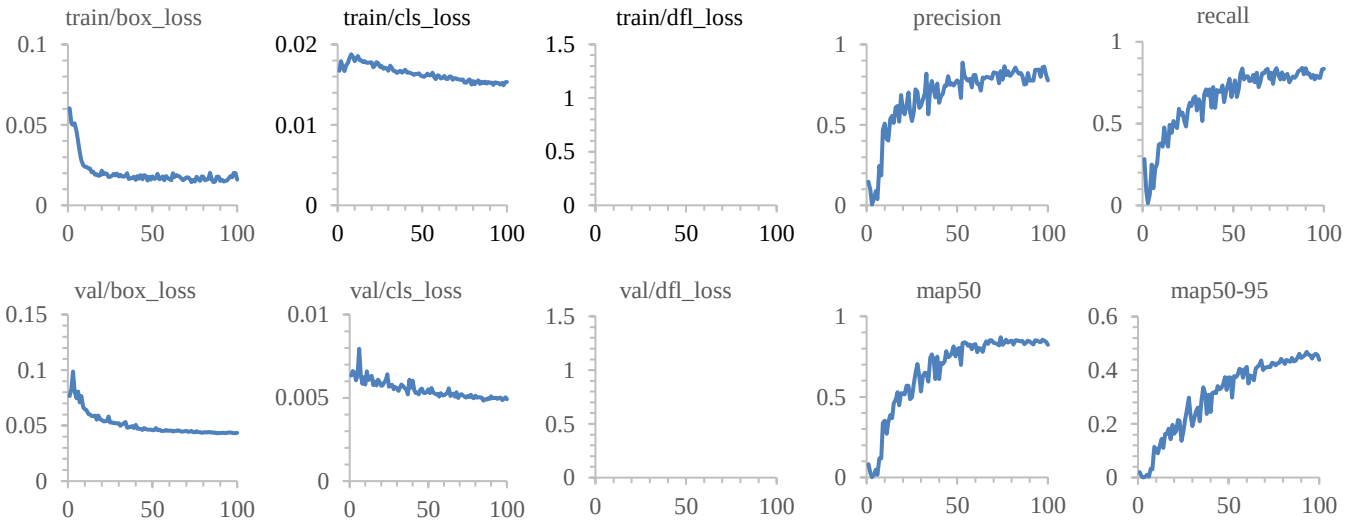

Figure 11. YOLOv7 training and validation results

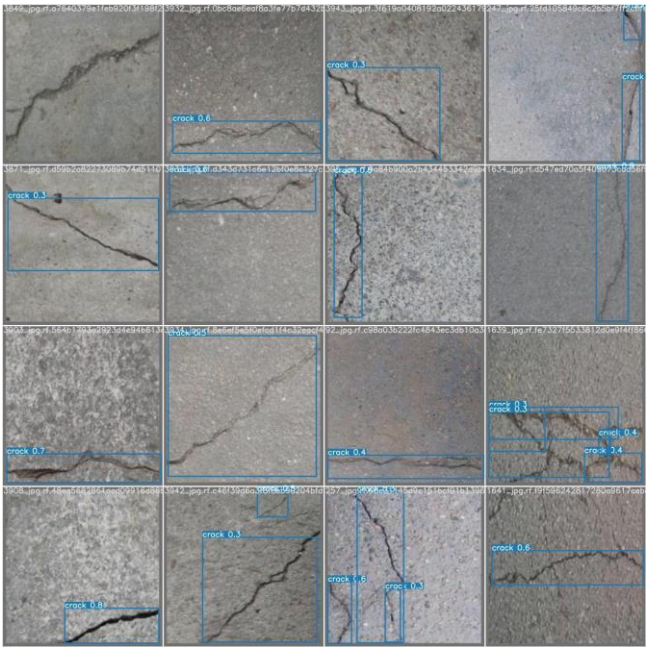

Figure 12. YOLOv7 result sample

Figure 12 shows sample of results for YOLOv7 model and how the model works in real-life.

4.4. YOLOv8 Results

The confusion matrix for YOLOv8 in Figure 13 reveals the number of instances that were correctly classified and misclassified for each class.

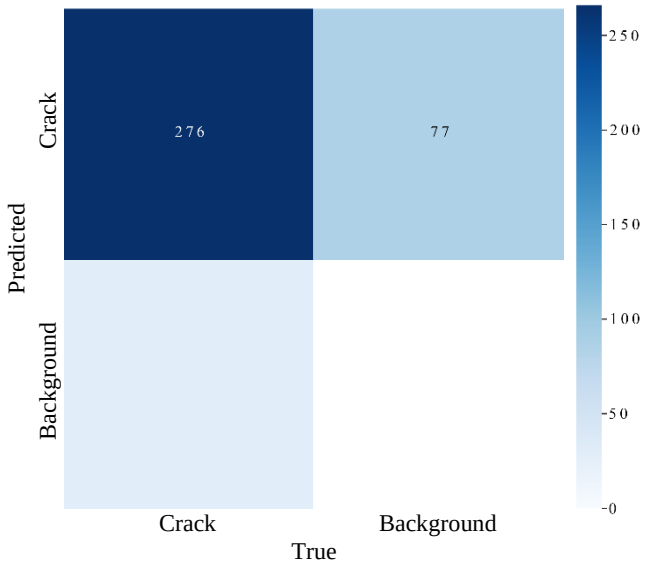

Figure 13. Confusion matrix for YOLOv8 model

Crack class has 276 instances correctly classified as Crack, and 77 instances misclassified as Background.

A detailed description of the YOLOv8 model's 100-epoch training procedure is shown in Figure 14. The model expedites the training process by effectively utilizing parallel processing capabilities. finished in the admirable time of 0.392 hours. The model performs admirably, as evidenced by the final metrics, which include a precision of 0.873, recall of 0.836, mAP50 of 0.898, and mAP50-95 of 0.645. Throughout the epochs, the model consistently reduced its loss values, culminating in final validation box, class, and DFL losses.

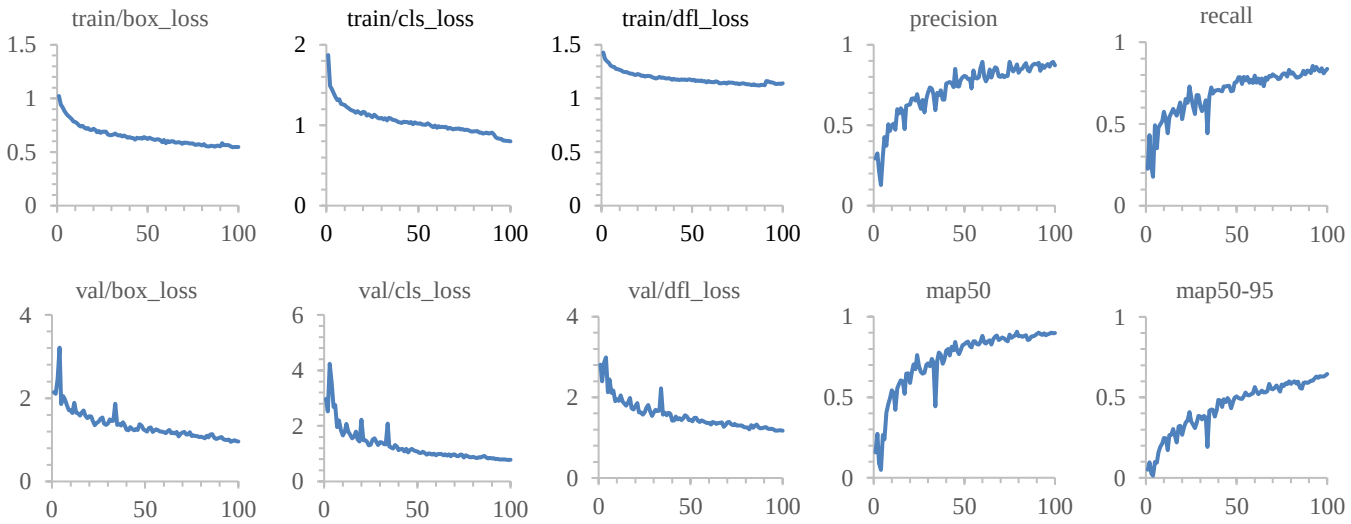

Figure 14. YOLOv8 training and validation results

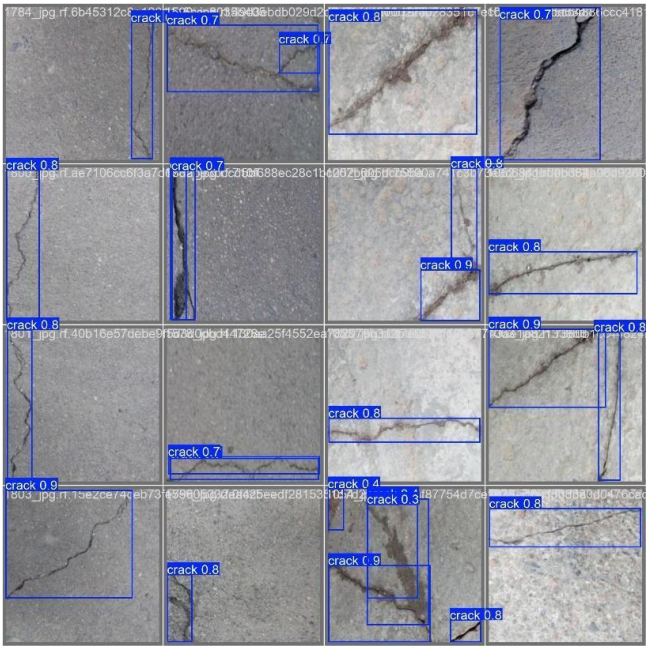

Figure 15. YOLOv8 result sample

Figure 15 shows sample of results for YOLOv8 model and how the model works in real-life.

4.5. YOLOv9 Results

The confusion matrix for YOLOv9 in Figure 16 reveals the number of instances that were correctly classified and misclassified for each class.

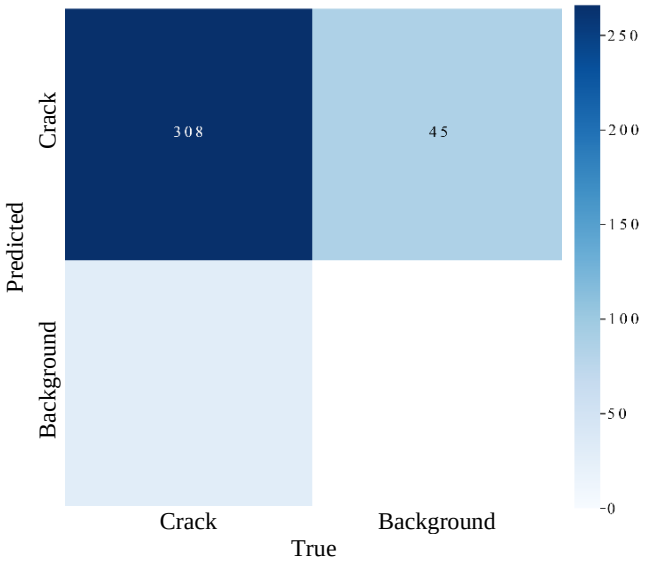

Figure 16. Confusion matrix for YOLOv9 model

Crack class has 308 instances correctly classified as Crack, and 45 instances misclassified as Background.

A detailed description of the YOLOv9 model's 100-epoch training procedure is shown in Figure 17. The model expedites the training process by effectively utilizing parallel processing capabilities. finished in the admirable time of 1.051 hours. The model performs admirably, as evidenced by the final metrics, which include a precision of 0.929, recall of 0.800, mAP50 of 0.921, and mAP50-95 of 0.721. Throughout the epochs, the model consistently reduced its loss values, culminating in final validation box, class, and DFL losses.

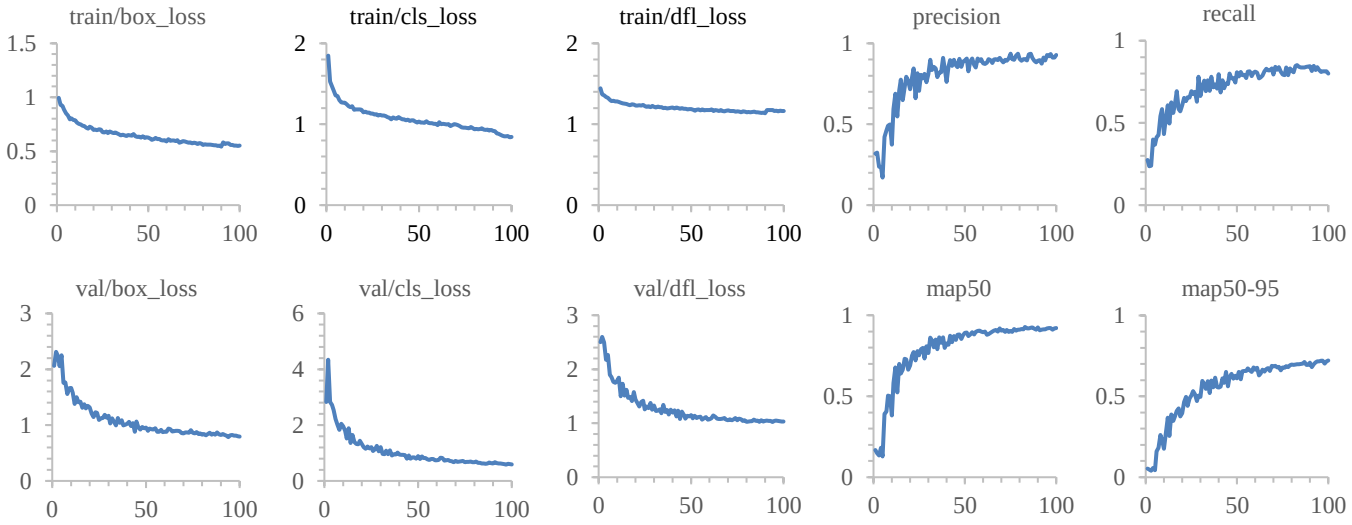

Figure 17. YOLOv9 training and validation results

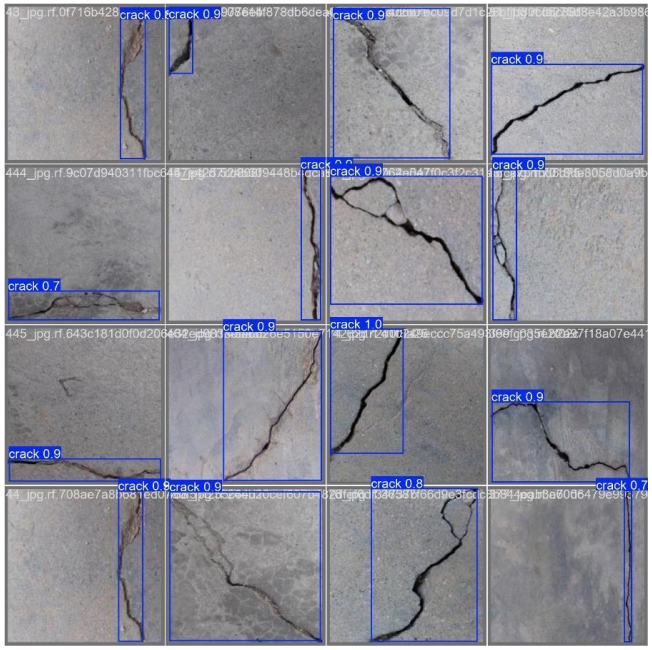

Figure 18. YOLOv9 result sample

Figure 18 shows sample of results for YOLOv9 model and how the model works in real-life.

4.6. YOLOv10 Results

The confusion matrix for YOLOv10 in Figure 19 reveals the number of instances that were correctly classified and misclassified for each class.

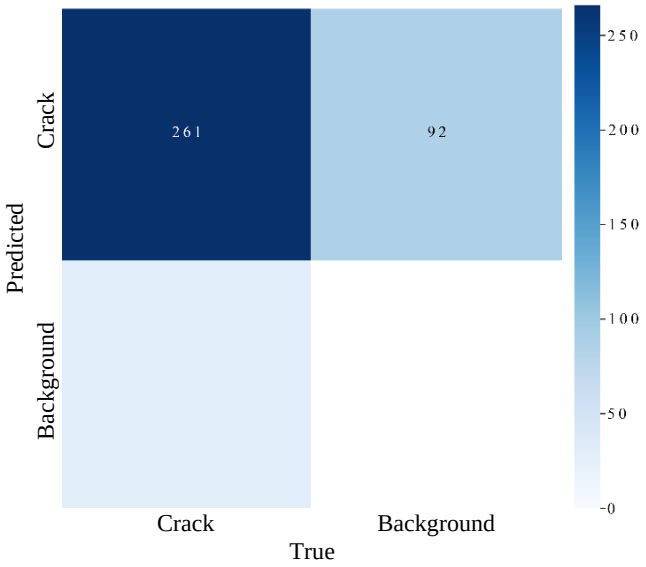

Figure 19. Confusion matrix for YOLOv10 model

Crack class has 261 instances correctly classified as Crack, and 92 instances misclassified as Background.

A detailed description of the YOLOv10 model's 100-epoch training procedure is shown in Figure 20. The model expedites the training process by effectively utilizing parallel processing capabilities. finished in the admirable time of 0.757 hours. The model performs admirably, as evidenced by the final metrics, which include a precision of 0.842, recall of 0.783, mAP50 of 0.860, and mAP50-95 of 0.612. Throughout the epochs, the model consistently reduced its loss values, culminating in final validation box, class, and DFL losses.

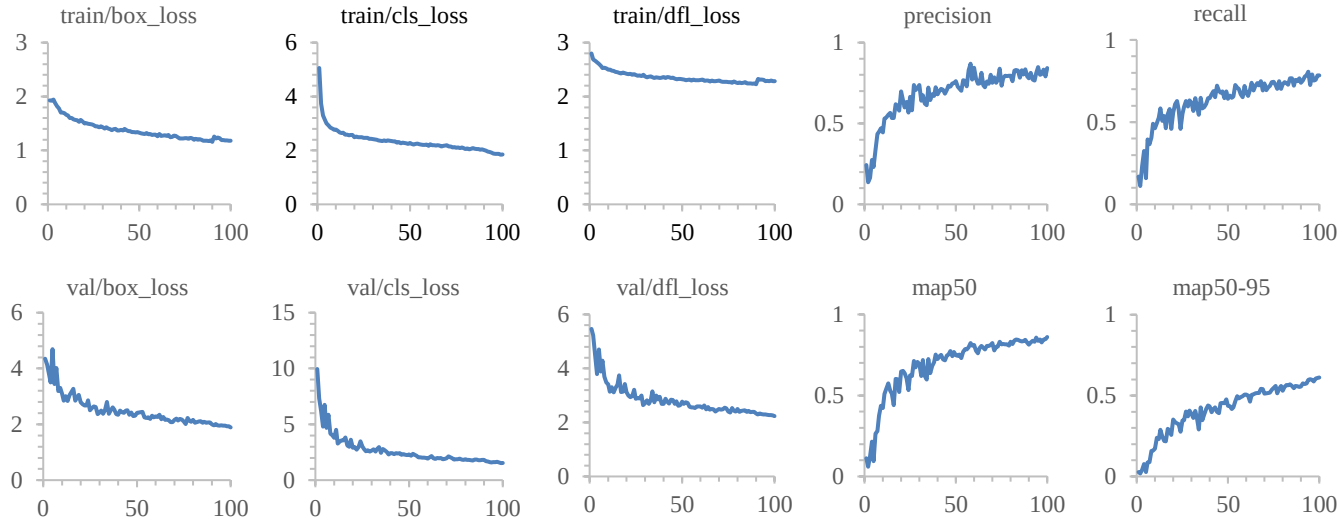

Figure 20. YOLOv10 training and validation results

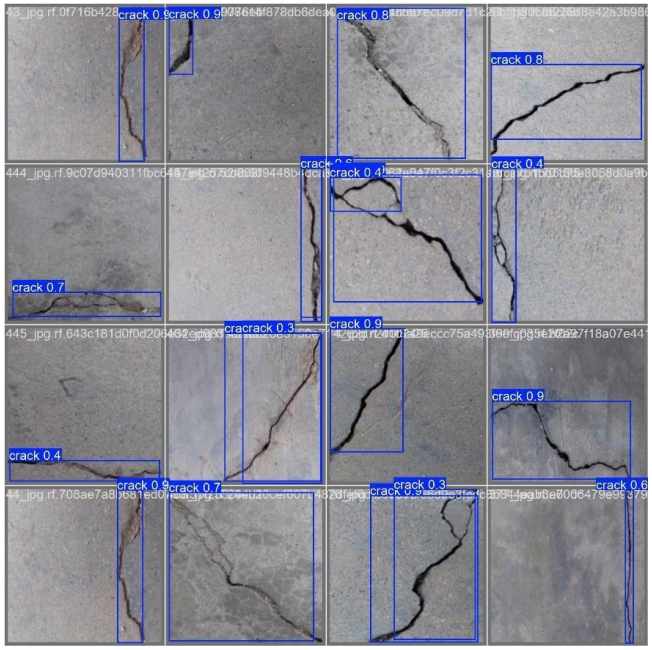

Figure 21. YOLOv10 result sample

Figure 21 shows sample of results for YOLOv10 model and how the model works in real-life.

4.7. Comparing between our YOLO models and other research

In a study[5], researchers studied on crack detection with the same dataset which was used in this study with YOLOv5 model. The summary of the comparison is given in Table 5.

Table 5. YOLOv5 comparing between other study and ours

Parameters	[5]	Our study
Model type	YOLOv5s	YOLOv5n
Training time	0.385	0.392
Model params	7.022 m	2.503 m
Input shape	416*416	640*640
Batch size	16	32
Learning rate	0.01	0.002
Optimizer	SGD	AdamW
Epochs	100	100
mAP50	0.862 avg.	0.903
mAP50-95	0.624 avg.	0.621
Precision	0.855 avg.	0.861
Recall	0.813	0.834
Environment	Google Colab	i9-14900K processor, 16GB RTX-A4000 GPU and 64GB RAM in Python-3.11.7.

The authors employed the YOLOv5 architecture to detect cracks in various masonry materials, including stone, brick, cob, tile, and concrete. The study aimed to develop a robust model for identifying cracks in cultural heritage structures, demonstrating the effectiveness of deep learning techniques in this domain [5]. When comparing the parameters of the YOLOv5 model used in this study with those from another study, as indicated in Table 5, several significant differences were found. Our study used the YOLOv5n variant, which has a smaller model size of 2.503 million parameters, whereas the other study used the YOLOv5s variant, which has 7.022 million parameters.

This study required slightly more training time (0.392 hours) than the other study (0.385 hours). Also in this study, a larger input size of 640x640 pixels were used, which can increase the ability of the model to capture more details. However, on the other study 416x416 pixels were used as input size. Similarly, the other study used a batch size of 16, while it was 32 in this study.

The learning rate and optimizer used in the two studies varied as well. The other study employed a learning rate of 0.01 with the SGD optimizer, while our study utilized a lower learning rate of 0.002 with the AdamW optimizer, which may contribute to different convergence behaviors during training.

The mAP50 score was 0.903 in this study, surpassing the 0.862 reported in the other study in terms of performance metrics. Additionally, the mAP50-95 score was 0.621 in this study, while it was 0.624 on the compared study.

Our study's precision score (0.861) was similar with the other study (0.855). Our study's recall was 0.834, which was higher than the other study's reported recall of 0.813.

Table 6. YOLOv7 comparing between other study and ours

Parameters	[1]	Our study
Model type	YOLOv7-custom	YOLOv7-custom
Training time	N/A	1.108
Model params	N/A	6.007 m
Input shape	416*416	640*640
Batch size	N/A	24
Learning rate	N/A	0.01
Epochs	120	100
mAP50	0.639 avg.	0.844
mAP50-95	0.35 avg.	0.466
Precision	0.722 avg.	0.836
Recall	0.608	0.776
Environment	N/A	i9-14900K processor, 16GB RTX-A4000 GPU and 64GB RAM with Python-3.11.7.

In a study, the authors used a YOLOv7-custom model to detect various types of damage in azulejos, the traditional tiles that adorn many historical structures in Portugal. This research highlights the significance of automating the inspection process to effectively preserve cultural heritage [1]. The comparison of the results was given in Table 6. When comparing the parameters and performance metrics, several key differences and similarities emerge. The other study employed a YOLOv7-custom model with an input shape of 416x416. In contrast, our study also utilized a YOLOv7-custom model but with a larger input size of 640x640 pixels and a batch size of 24, which may enhance the model's ability to capture more detailed features in the images.

Our study required 1.108 hours of training, while the other study did not specify a time frame. There was also a difference in the quantity of epochs used; our study involved 100 epochs, while the other study trained for 120 epochs. The other study did not include this information, which might have affected the model's convergence and overall performance.

The mAP50 score was found as 0.844 in this study, while it was found as 0.639 in terms of performance metrics. Our study reported a mAP50-95 of 0.466, whereas the other study

had 0.350 value. Furthermore, our recall score was higher at 0.776, compared to the other study's recall score of 0.608, and our precision score was recorded at 0.836, as opposed to 0.722 in that study.

The computational environment for our study included an i9-14900K processor, 16GB RTX-A4000 GPU, and 64GB RAM, which likely contributed to the improved performance metrics observed. However, the hardware specifications were not clearly given in other study (Pytorch library in Google Colaboratory (COLAB) with the available GPU memory (NVIDIA Tesla T4, 16GB).).

Both studies contribute valuable insights into the application of YOLOv7 for detecting defects in cultural heritage materials. Our study demonstrates better performance metrics, including higher mAP, precision, and recall scores, indicating more effective crack detection across various materials compared to the tile defect detection study.

In another study[40], YOLOv5 and YOLOv8 models were used for crack detection. The data resources were completely different with our study. When comparing the models, it was found that only the precision value was lower than other study for YOLOv5, but mAP50, Recall and F1-Score were higher [40].

5. Conclusion

This study investigated the effectiveness of various YOLO algorithms for detecting surface cracks in historical buildings. Among the evaluated algorithms, YOLOv9 emerged as the most accurate and effective model, demonstrating the best performance in both accuracy and speed measurements. This study highlights the potential of deep learning-based methods, particularly YOLO algorithms, to significantly enhance the conservation efforts of cultural heritage sites through reliable, automatic crack detection.

The comparative analysis of YOLOv5, YOLOv6, YOLOv7, YOLOv8, YOLOv9, and YOLOv10 revealed significant improvements in each iteration, with YOLOv9 standing out with its balance between computational efficiency and detection accuracy achieving a mAP50 score of 0.921 and a mAP50-95 score of 0.721 for the given dataset. YOLOv8 performed well scores with a mAP50 score of 0.898 and comes in second place. On the other hand, YOLOv5 and YOLOv7 exhibited lower accuracy, especially in the mAP50-95 range. The results highlight the progress made in YOLO algorithms and the importance of selecting the right model based on the needs.

Future research should focus on further optimizing these models for real-time applications in various environmental conditions, as well as expanding the dataset to include a wider variety of materials and structural types. In addition, integrating these detection systems with broader structural health monitoring frameworks can provide more comprehensive solutions for the preservation of historic buildings and ensure their longevity and structural integrity for future generations.

Conflict of Interest Statement

The authors declare no conflict of interest.

References

[1] N. Karimi, M. Mishra, P.B. Lourenço, Deep learning-based automated tile defect detection system for Portuguese cultural heritage buildings, Journal of Cultural Heritage 68 (2024) 86–98. https://doi.org/10.1016/j.culher.2024.05.009.

[2] P.B. Lourenço, M.P. Ciocci, F. Greco, G. Karanikoloudis, C. Cancino, D. Torrealva, K. Wong, Traditional techniques for the rehabilitation and protection of historic earthen structures: The seismic retrofitting project, International Journal of Architectural Heritage 13(1) (2019) 15–32. https://doi.org/10.1080/15583058.2018.1497232.

[3] W. Błaszczak-Bąk, C. Suchocki, J. Janicka, A. Dumalski, R. Duchnowski, A. Sobieraj-Żłobińska, Automatic threat detection for historic buildings in dark places based on the modified OPTD method, ISPRS International Journal of Geo-Information 9(2) (2020) 123. https://doi.org/10.3390/ijgi9020123.

[4] B. Tejedor, E. Lucchi, D. Bienvenido-Huertas, I. Nardi, Non-destructive techniques (NDT) for the diagnosis of heritage buildings: Traditional procedures and futures perspectives, Energy and Buildings 263 (2022) 112029. https://doi.org/10.1016/j.enbuild.2022.112029.

[5] N. Karimi, M. Mishra, P.B. Lourenço, Automated Surface Crack Detection in Historical Constructions with Various Materials Using Deep Learning-Based YOLO Network, International Journal of Architectural Heritage (2024) 1–17. https://doi.org/10.1080/15583058.2024.2376177.

[6] G. Stephen, J. Brownjohn, C. Taylor, Measurements of static and dynamic displacement from visual monitoring of the Humber Bridge, Engineering Structures 15(3) (1993) 197-208. https://doi.org/10.1016/0141-0296(93)90054-8.

[7] I. Abdel-Qader, O. Abudayyeh, M.E. Kelly, Analysis of edge-detection techniques for crack identification in bridges, Journal of computing in civil engineering 17(4) (2003) 255–263. https://doi.org/10.1061/(ASCE)0887-3801(2003)17:4(255).

[8] Y.-J. Cha, K. You, W. Choi, Vision-based detection of loosened bolts using the Hough transform and support vector machines, Automation in Construction 71 (2016) 181-188. https://doi.org/10.1016/j.autcon.2016.06.008.

[9] S. Patsias, W. Staszewskiy, Damage detection using optical measurements and wavelets, Structural Health Monitoring 1(1) (2002) 5-22. https://doi.org/10.1177/147592170200100102.

[10] M. Gkantou, M. Muradov, G.S. Kamaris, K. Hashim, W. Atherton, P. Kot, Novel electromagnetic sensors embedded in reinforced concrete beams for crack detection, Sensors 19(23) (2019) 5175. https://doi.org/10.3390/s19235175.

[11] K.L. Rens, T.J. Wipf, F.W. Klaiber, Review of nondestructive evaluation techniques of civil infrastructure, Journal of performance of constructed facilities 11(4) (1997) 152-160. https://doi.org/10.1061/(ASCE)0887-3828(1997)11:4(152).

[12] K. Jang, N. Kim, Y.-K. An, Deep learning–based autonomous concrete crack evaluation through hybrid image scanning, Structural Health Monitoring 18(5-6) (2019) 1722–1737. https://doi.org/10.1177/1475921718821719.

[13] C. Zhang, Z. Alam, L. Sun, Z. Su, B. Samali, Fibre Bragg grating sensor-based damage response monitoring of an asymmetric reinforced concrete shear wall structure subjected to progressive seismic loads, Structural Control and Health Monitoring 26(3) (2019) e2307. https://doi.org/10.1002/stc.2307.

[14] S. Fan, L. Ren, J. Chen, Investigation of fiber Bragg grating strain sensor in dynamic tests of small-scale dam model, Structural Control and Health Monitoring 22(10) (2015) 1282–1293. https://doi.org/10.1002/stc.1745.

[15] S. Acikgoz, L. Pelecanos, G. Giardina, J. Aitken, K. Soga, Distributed sensing of a masonry vault during nearby piling, Structural Control and Health Monitoring 24(3) (2017) e1872. https://doi.org/10.1002/stc.1872.

[16] C.M. Yeum, S.J. Dyke, Vision-based automated crack detection for bridge inspection, Computer-Aided Civil and Infrastructure Engineering 30(10) (2015) 759-770. https://doi.org/10.1111/mice.12141.

[17] Y. Xu, S. Li, D. Zhang, Y. Jin, F. Zhang, N. Li, H. Li, Identification framework for cracks on a steel structure surface by a restricted Boltzmann machines algorithm based on consumer-grade camera images, Structural Control and Health Monitoring 25(2) (2018) e2075. https://doi.org/10.1002/stc.2075.

[18] M.R. Valluzzi, L. Sbrogiò, Y. Saretta, H. Wenliuhan, Seismic response of masonry buildings in historical centres Struck by the 2016 central Italy earthquake. Impact of building features on damage evaluation, International Journal of Architectural Heritage 16(12) (2022) 1859–1884. https://doi.org/10.1080/15583058.2021.1916852.

[19] N. Davies, E. Jokiniemi, Dictionary of architecture and building construction, Routledge2008. https://doi.org/10.4324/9780080878744.

[20] A.M.A. Talab, Z. Huang, F. Xi, L. HaiMing, Detection crack in image using Otsu method and multiple filtering in image processing techniques, Optik 127(3) (2016) 1030-1033. https://doi.org/10.1016/j.ijleo.2015.09.147.

[21] R. Adhikari, O. Moselhi, A. Bagchi, Image-based retrieval of concrete crack properties for bridge inspection, Automation in construction 39 (2014) 180–194. https://doi.org/10.1016/j.autcon.2013.06.011.

[22] N. Wang, X. Zhao, L. Wang, Z. Zou, Novel system for rapid investigation and damage detection in cultural heritage conservation based on deep learning, Journal of Infrastructure Systems 25(3) (2019) 04019020. https://doi.org/10.1061/(ASCE)IS.1943-555X.0000499.

[23] A. Tavukçuoğlu, S. Akevren, E. Grinzato, In situ examination of structural cracks at historic masonry structures by quantitative infrared thermography and ultrasonic testing, Journal of Modern Optics 57(18) (2010) 1779–1789. https://doi.org/10.1080/09500340.2010.484553.

[24] G. Pascale, A. Lolli, Crack assessment in marble sculptures using ultrasonic measurements: Laboratory tests and application on the statue of David by Michelangelo, Journal of Cultural Heritage 16(6) (2015) 813-821. https://doi.org/10.1016/j.culher.2015.02.005.

[25] S. Pratibha, N. Dhananjaya, Phytobial remediation: a new technique for ecological sustainability, Agricultural and Environmental Nanotechnology: Novel Technologies and their Ecological Impact, Springer2023, 451-462. https://doi.org/10.1007/978-981-19-5454-2_17.

[26] Y. Zhao, Z. Ju, T. Sun, F. Dong, J. Li, R. Yang, Q. Fu, C. Lian, P. Shan, Tgc-yolov5: An enhanced yolov5 drone detection model based on transformer, gam & ca attention mechanism, Drones 7(7) (2023) 446. https://doi.org/10.3390/drones7070446.

[27] V.A. Adibhatla, H.-C. Chih, C.-C. Hsu, J. Cheng, M.F. Abbod, J.-S. Shieh, Applying deep learning to defect detection in printed circuit boards via a newest model of you-only-look-once, Math Biosci Eng 18(4) (2021) 4411–4428. https://doi.org/10.3934/mbe.2021223.

[28] Z. Wang, H. Zhang, Z. Lin, X. Tan, B. Zhou, Prohibited items detection in baggage security based on improved YOLOv5, 2022 IEEE 2nd international conference on software engineering and artificial intelligence (SEAI), IEEE, 2022, pp. 20-25.

[29] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 779-788.

[30] M.E. Atik, Z. Duran, R. Özgünlük, Comparison of YOLO versions for object detection from aerial images, International journal of environment and geoinformatics 9(2) (2022) 87–93. https://doi.org/10.30897/ijegeo.1010741.

[31] U. Sirisha, S.P. Praveen, P.N. Srinivasu, P. Barsocchi, A.K. Bhoi, Statistical analysis of design aspects of various YOLO-based deep learning models for object detection, International Journal of Computational Intelligence Systems 16(1) (2023) 126. https://doi.org/10.1007/s44196-023-00302-w.

[32] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, G. Ding, Yolov10: Real-time end-to-end object detection, arXiv preprint arXiv:2405.14458 (2024). https://doi.org/10.48550/arXiv.2405.14458.

[33] S.-J. Hong, S.-Y. Kim, E. Kim, C.-H. Lee, J.-S. Lee, D.-S. Lee, J. Bang, G. Kim, Moth detection from pheromone trap images using deep learning object detectors, Agriculture 10(5) (2020) 170. https://doi.org/10.3390/agriculture10050170.

[34] G. Yu, X. Zhou, An improved YOLOv5 crack detection method combined with a bottleneck transformer, Mathematics 11(10) (2023) 2377. https://doi.org/10.3390/math11102377.

[35] N.I.M. Yusof, A. Sophian, H.F.M. Zaki, A.A. Bawono, A. Ashraf, Assessing the performance of YOLOv5, YOLOv6, and YOLOv7 in road defect detection and classification: a comparative study, Bulletin of Electrical Engineering and Informatics 13(1) (2024) 350–360. https://doi.org/10.11591/eei.v13i1.6317.

[36] J. Xing, Y. Liu, G.-Z. Zhang, Improved yolov5-based uav pavement crack detection, IEEE Sensors Journal 23(14) (2023) 15901-15909. https://doi.org/10.1109/JSEN.2023.3281585.

[37] H. Wang, X. Han, X. Song, J. Su, Y. Li, W. Zheng, X. Wu, Research on automatic pavement crack identification Based on improved YOLOv8, International Journal on Interactive Design and Manufacturing (IJIDeM) (2024) 1–11. https://doi.org/10.1007/s12008-024-01769-3.

[38] X. Chen, C. Wang, C. Liu, X. Zhu, Y. Zhang, T. Luo, J. Zhang, Autonomous crack detection for mountainous roads using UAV inspection system, Sensors 24(14) (2024) 4751. https://doi.org/10.3390/s24144751.

[39] S. Youwai, A. Chaiyaphat, P. Chaipetch, YOLO9tr: A Lightweight Model for Pavement Damage Detection Utilizing a Generalized Efficient Layer Aggregation Network and Attention Mechanism, arXiv preprint arXiv:2406.11254 (2024). https://doi.org/10.48550/arXiv.2406.11254.

[40] S. Mohanty, S.K. Pani, Empowering Structural Integrity: YOLO-Based Crack Detection and MLDriven Concrete Strength Prediction of Critical Infrastructure Caused due to Mining Operation, Journal of Electrical Systems 20(5s) (2024) 2705-2721. https://doi.org/10.52783/jes.2752.